



madhumitha@localhost:~



1. Producer-Consumer Problem using Semaphores - producer_consumer.c

```
[madhumitha@localhost ~]$ cat producer_consumer.c
```

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full, mutex;

void *producer(void *arg)
{
    int item;
    while (1)
    {
        item = rand() % 100;
        sem_wait(&empty);
```



madhumitha@localhost:~



2. Compile the Program

```
[madhumitha@localhost ~]$ gcc producer_consumer.c -o producer_consumer  
-lpthread
```

```
[madhumitha@localhost ~]$
```

```
[madhumitha@localhost ~]$ ./producer_consumer
```

```
Producer produced: 83
```

```
Consumer consumed: 83
```

```
Producer produced: 86
```

```
Consumer consumed: 86
```

```
Producer produced: 77
```

```
Consumer consumed: 77
```

```
Producer produced: 15
```

```
Consumer consumed: 15
```

```
Producer produced: 93
```

```
Consumer consumed: 93
```

```
Producer produced: 35
```

```
Consumer consumed: 35
```

```
Producer produced: 86
```

```
Consumer consumed: 86
```

```
Producer produced: 92
```

```
Consumer consumed: 92
```

```
Producer produced: 49
```

```
Consumer consumed: 49
```

```
...
```



4. Explanation

This program demonstrates the classic Producer-Consumer problem using POSIX semaphores.

Semaphores used:

1. **empty** -> counts empty slots in the buffer
2. **full** -> counts filled slots in the buffer
3. **mutex** -> provides mutual exclusion while accessing buffer

Buffer Size: 5

Producer produces random items and inserts into the buffer.

Consumer consumes items from the buffer.

The semaphores ensure proper synchronization between producer and consumer threads.

```
[madhumitha@localhost ~]$
```